

Escuela de Administración de Tecnologías de Información

Bases de Datos – TI3600

Proyecto #2
Licitación para ecosistema Digital de Cines IMAX

Estudiantes:
Victoria Alvarado Rivera
Gabriel Gutiérrez Brenes

Profesora:
Ing. Jacqueline Solís Céspedes

II Semestre

2026

Índice

Desarrollo **3**
1. Dictamen de rechazo y propuesta de factibilidad 3
1.1. Propuesta 3
1.2. Horizonte de evaluación (proyección financiera a 3 años) 4
1.3. Comparativa de proveedores 4
Alternativas (Google Cloud y 4
2. Análisis de requisitos y dimensionamiento de datos 5
2.1. Requerimientos funcionales: 5
2.2. Requerimientos no funcionales 5
3. Diseños 6
3.1 Modelo conceptual: 9
3.2 Modelo lógico 10
4. Modelo Relacional 16
5. Interfaz gráfica 17
5.1 Vista de cartelera y exploración 17
5.2 Mapa interactivo de la sala 17
5.3 Módulo de pago 17
5.4 Pantalla de confirmación y entrada digital 18
5.5 Panel de administración (dashboard) 18
5.5.1. Validación de QR 18
5.5.2. Administración 19
5.5.3. Reportes 19
Bibliografía **21**
Anexos **22**

Desarrollo

1. Dictamen de rechazo y propuesta de factibilidad

El sistema legado no soportaría la concurrencia esperada ya que la única instancia no permite que el servidor pueda tener un crecimiento exponencial. Este componente ocasiona cuellos de botella debidos a sus limitadas capacidades computacionales (como su límite de CPU, RAM y un ancho de banda limitado por las velocidades de los buses internos).

Por otro lado, el no uso de infraestructura en nube también representa una gran amenaza y problema ya que implica que el servidor está contenido en una máquina con componentes deficientes y anticuados. Almacenar el servidor en el equipo actual no permite un crecimiento de datos ni de concurrencia.

Además, guardar 8 millones de compras de tickets de manera “manual” por parte de los usuarios es ineficiente ya que, si un usuario quiere comprar más de una entrada, debe hacer cada entrada individualmente, esto vuelve el proceso lento, tedioso y desconfiable. Esta forma de guardar registros no es confiable y compromete su trazabilidad; el cliente puede equivocarse al introducir sus datos y no tiene manera de saber si está equivocado. Además, se pueden manipular los datos de todos los clientes de forma sencilla, esto hace que la trazabilidad de los datos sea difícil.

El trato que se le da a los datos bancarios es altamente negligente. Esta información debería ser de carácter confidencial y no debería ser transmitida por un canal tan vulnerable como lo es un WhatsApp corporativo. Adicionalmente, no es solo negligente, también es sumamente ineficiente: con una pasarela de pago solo se tendrían que introducir los datos de la tarjeta y se aplica el pago pero en el modelo actual propone agregar tres pasados adicionales que son inseguros y propensos a errores con información sensible.

El respaldo se vuelve más concreto si se concreta un servicio en la nube, ya que, ante una posible caída de servidor, este servicio puede seguir trabajando al poco tiempo; este servicio tiene una serie de pasos y defensas técnicas para proteger los datos en caso de algún percance.

Por las razones anteriormente mencionadas, esta propuesta debe ser descartada de manera inmediata. La propuesta no permite que sea auditable, no tiene procesos transparentes; controles definidos; o accesos claros y rastreables. Además no permite que el servidor pueda crecer para soportar el pico de demanda, ni tampoco es un modelo de operación sustentable a corto plazo.

1.1. Propuesta

Proponemos usar un conjunto de servicios en la nube para garantizar el buen funcionamiento del servicio de las ventas de boletos. El sistema a usar es Amazon Web Services (AWS), la implementación de este sistema permitirá soportar picos, miles de transacciones por segundo, procesamiento de registros para una auditoría y más.

Se propone usar AWS Lambda para poder escalar servidores según demanda; AWS WAF y API Gateway para proteger los datos del usuario; Amazon Aurora para el procesamiento de transacciones bancarias; PostgreSQL para tener un almacén de datos y Amazon RDS Proxy para no sobresaturar la base de datos con el volumen de transacciones.

Con esta propuesta el equipo computacional legado puede ser desechado ya que deja de ser el “servidor principal” del servicio de boletos. Sería beneficioso que estos equipos sean actualizados para ser compatibles con la versión más actual de los sistemas necesarios para la correcta operación de la propuesta, pero sí se recomienda actualizar la conexión residencial ya que esta no es apropiada para los niveles de transacciones. Se debe sustituir por una conexión empresarial de fibra óptica que tenga acceso directo a los diversos servidores de AWS.

Al aplicar una pasarela de pagos, esta transacción se hace de manera automática, confidencial y encriptada. Una vez que el pago se procese, la información del monto, persona, cantidad de tiquetes, función, referencia y fecha se almacenan dentro de la base de datos. De esta manera el margen de error es casi nulo y los datos sensibles del cliente no se almacenan.

1.2. Horizonte de evaluación (proyección financiera a 3 años)

Año	Opción #1: AWS (Amazon Web Services)		Opción #2: Microsoft Azure	
	CAPEX	**OPEX**	**CAPEX**	**OPEX**
1	$24 000	$288 000	$22 850	$274 200
2	+ 10%	$316 800	+ 10%	$301 620
3	+ 10%	$348 480	+ 10%	$331 782
COSTO TOTAL	**Total OPEX**	**$953 280**	**Total OPEX**	**$907 602**

1.3. Comparativa de proveedores
Alternativas (Google Cloud y

Componente arquitectónico	Opción #1: AWS (Amazon Web Services)	Opción #2: Microsoft Azure	Opción #3: Google Cloud
Base de datos	$12 500	$11 800	$115.62
Cómputo	$6 200	$5 900	$67.01
Almacenamiento	$1 800	$1 750	$99.9
Seguridad	$3 500	$3 400	$99 703.58
TOTAL	**$24 000 por mes**	**$22 850 por mes**	**$99 986.12 por mes**

De acuerdo a los datos de una tercera opción, se puede identificar que es una opción más cara en cuanto a la implementación de seguridad al sistema. De acuerdo a la calculadora de precios de Google Cloud, el componente para asegurar la seguridad es el *Global Front End*, que se puede calcular por medio de la región la cuál se procesa. Los demás componentes son *Compute Engine* para el cómputo; *Cloud SQL* para la base de datos y *Cloud Storage* para el almacenamiento. Aunque se modifiquen los requerimientos y cantidad de datos en Google Cloud, la opción más rentable corresponde a la #1: AWS (Amazon Web Services).

2. Análisis de requisitos y dimensionamiento de datos

2.1. Requerimientos funcionales:

Código	Requerimiento	Descripción
RF-01	Cartelera de películas disponibles	La interfaz del sistema debe mostrar las películas cuya fecha sea mayor a la actual. Si no están disponibles, no se muestran.
RF-02	Mapa de asientos	Los asientos se deben mostrar en forma de cuadrícula para poder ver los disponibles y los ocupados. Se muestran de acuerdo al atributo 'estado' de la entidad 'asiento'.
RF-03	Módulo de pago de entrada	Dentro de la pestaña de clientes, se debe mostrar un botón de pago (simulación de pago)
RF-04	Comprobante de entrada	Luego de seleccionar y realizar el pago, el sistema debe mostrar un mensaje como comprobante del pago y compra de la entrada.
RF-05	Panel administrativo (*dashboard*)	El sistema debe mostrar una pestaña llamada "Dashboard" que solicite un inicio de sesión a las personas internas que pueden gestionar el sistema: administradores, operadores, contadores y taquilleros.

2.2. Requerimientos no funcionales

Código	Requerimiento	Descripción
RNF-01	Pestañas para cada sección de operaciones dentro del sistema	El sistema debe componerse de cuatro pestañas que identifiquen y contengan las secciones de operación y visualización de datos: clientes (agregar), dashboard (administración y operaciones de acuerdo al rol: administrador
RNF-02	Visualización de cartelera	La cartelera de funciones debe mostrar la información completa de las películas disponibles por orden fecha (si la fecha es pasada, no se muestra): nombre y fecha.
RNF-03	Pago de cliente (simulación)	La opción de pago debe mostrar una opción de texto para ingresar el nombre del cliente y simular una contraseña bancaria con el id. Esto permite realizar la transacción y generar el comprobante.
RNF-04	Comprobante de entrada	El comprobante de compra debe presentar como mínimo un mensaje de validación: debe incluir el nombre del cliente, tarifa, id de la entrada, nombre de película y asiento comprado.

3. <u>Diseños</u>

Identificación de entidades:

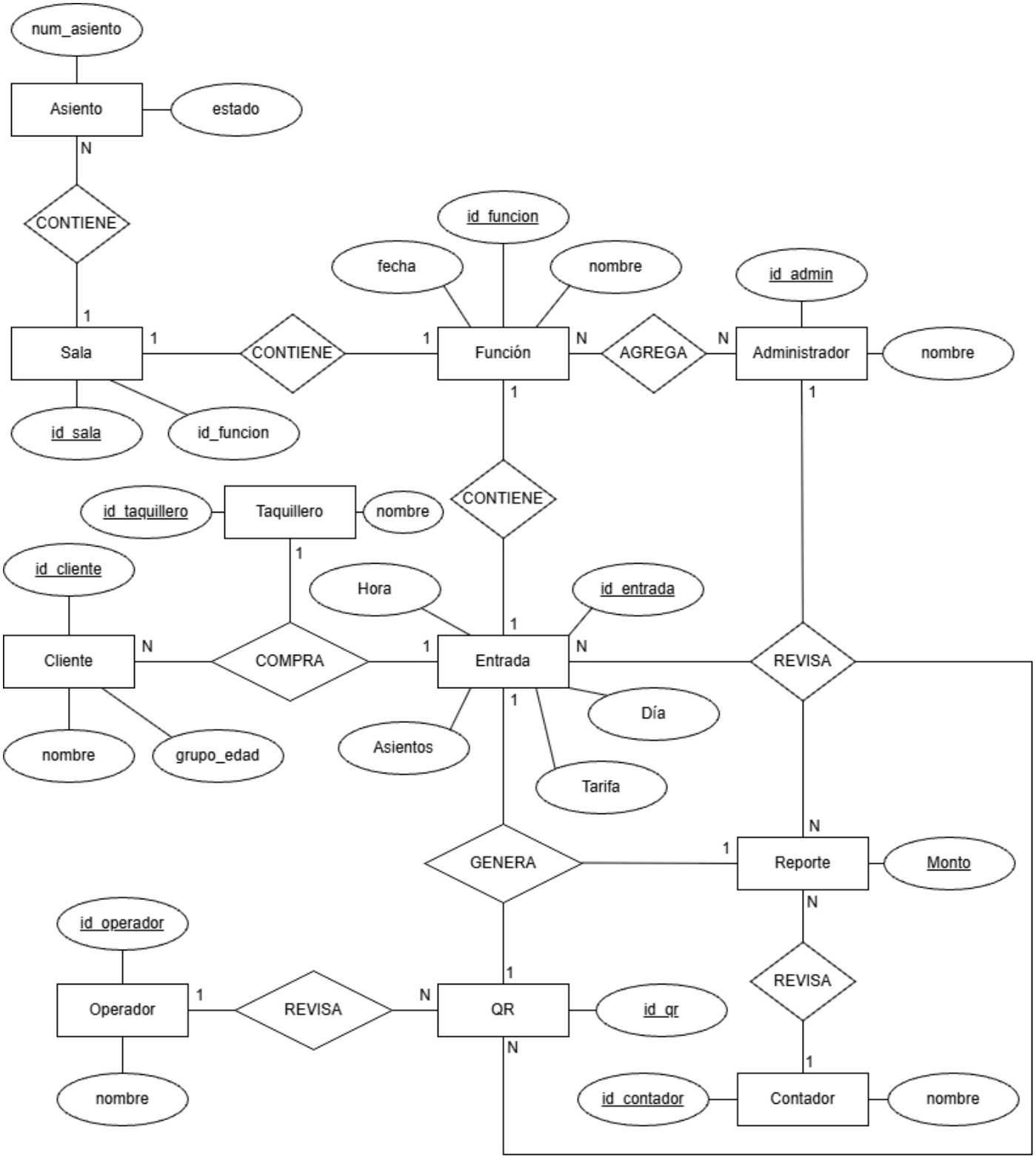
Entidad	Atributos	Datos atributo	Operaciones
Cliente	id_cliente [PK]	id numérico	El cliente es quien compra las entradas, selecciona los asientos y realiza el pago correspondiente
	nombre		
	grupo_edad	(adulto, niño, mayor o estudiante)	
Entrada	función [PK]	nombre película	La entrada es el producto que compra el cliente. El proceso de compra genera un código QR como comprobante. Se relaciona el id_taquillero como FK para identificar quién realizó la venta
	asientos	número	
	día		
	hora		
	tarifa	monto total (promoción: si es 0 no hay descuento; se calcula de acuerdo al atributo grupo_edad de la entidad Cliente)	
	id_taquillero [FK]	id numérico	
Taquillero	id_taquillero [PK]	id numérico	El taquillero se encarga de realizar la venta de entradas a clientes
	nombre		
Operador	id_operador [PK]	id numérico	El operador es quien valida el QR de la compra de clientes
	nombre		
Contador	id_contador [PK]	id numérico	El contador genera reportes financieros y tiene acceso a los montos por compras y totales
	nombre		
Administrador	id_admin [PK]	id numérico	El administrador tiene acceso total a todas las funciones y departamentos
	nombre		
Comprobante	id_qr [PK]	id numérico	Corresponde al QR que comprueba el operador como compra de una entrada
Reporte	No hay		El contador puede ver este reporte como datos por venta o monto total de ventas. Corresponde a una consulta SQL.
Función	id_funcion [PK]	id numérico	La función está contenida dentro de la entrada para identificar cuál

Entidad	Atributos	Datos atributo	Operaciones
	fecha		película verá el cliente. Son añadidas por los administrativos.
	nombre		
Sala	id_sala [PK]	id numérico	La sala es el lugar donde se proyecta la función, por lo que tiene un id para identificarlo y una llave foránea hacia la función la cual se proyectará
	id_funcion [FK]	id numérico	
Asiento	num_asiento [PK]	número del asiento que se compra	El asiento es donde estará el cliente en el momento de ver la función. Se compone de un número, estado (libre u ocupado) y una llave foránea a la sala donde se encuentra
	estado	libre u ocupado	
	id_sala [FK]	id numérico	

Identificación de relaciones

Nombre y Tipo	Entidades	Cardinalidad	Descripción
COMPRA (N-aria)	Cliente	N	N clientes pueden comprar 1 entrada, a una función, vendida por 1 taquillero
	Entrada	1	
	Taquillero	1	
GENERA (N-aria)	Entrada	1	1 entrada genera 1 comprobante y 1 reporte
	Comprobante	1	
	Reporte	1	
REVISA (Binaria)	Contador	1	1 contador revisa N reportes de ventas
	Reporte	N	
REVISA (Binaria)	Operador	1	1 operador revisa N comprobantes de ventas
	Comprobante	N	
REVISA (N-aria)	Administrador	1	1 Administrador puede revisar N entradas, reportes y comprobantes (acceso total)
	Entrada	N	
	Reporte	N	
	Comprobante	N	
CONTIENE (Binaria)	Entrada	1	1 entrada contiene 1 función (la que verá el cliente)
	Función	1	
AGREGA (Binaria)	Administrador	N	N administradores pueden agregar N cantidad de funciones al sistema
	Función	N	
CONTIENE (Binaria)	Función	1	1 función contiene 1 sala (donde se proyectará la función)
	Sala	1	
CONTIENE (Binaria)	Sala	1	1 sala contiene N cantidad de asientos (donde estarán los clientes al ver la función)
	Asiento	N	

3.1 Modelo conceptual:



3.2 Modelo lógico

```
cliente (id_cliente [PK], nombre, grupo_edad)
funcion (id_funcion [PK], nombre, fecha)
entrada (id_entrada [PK], función, asientos, día, hora, tarifa, id_taquillero [FK])
taquillero (id_taquillero [PK], nombre)
operador (id_operador [PK], nombre)
contador (id_contador [PK], nombre)
administrador (id_admin [PK], nombre)
comprobante (id_qr [PK])
sala (id_sala [PK], id_función [FK])
asiento (num_asiento [PK], estado, id_sala [FK])
```

3.3.1 Creación de tablas:

```
CREATE TABLE cliente (
    id_cliente BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    grupo_edad VARCHAR(15) NOT NULL CHECK (grupo_edad IN
('adulto','niño','mayor','estudiante'))
);
CREATE TABLE taquillero (
    id_taquillero BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);
CREATE TABLE operador (
    id_operador BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);
CREATE TABLE contador (
    id_contador BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);
CREATE TABLE administrador (
    id_admin BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);
CREATE TABLE pelicula (
    id_pelicula BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    sinopsis TEXT NOT NULL,
    clasificacion VARCHAR(20) NOT NULL,
    duracion_min INT NOT NULL CHECK (duracion_min > 0),
    activa BOOLEAN NOT NULL DEFAULT TRUE
);
CREATE TABLE sala (
    id_sala BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    ubicacion VARCHAR(200) NOT NULL,
    capacidad INT NOT NULL CHECK (capacidad > 0)
```

```
);
CREATE TABLE asiento (
    id_sala BIGINT NOT NULL REFERENCES sala(id_sala),
    numero VARCHAR(12) NOT NULL,
    zona VARCHAR(30) NOT NULL DEFAULT 'general',
    habilitado BOOLEAN NOT NULL DEFAULT TRUE,
    PRIMARY KEY (id_sala, numero)
);
CREATE TABLE funcion (
    id_funcion BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_pelicula BIGINT NOT NULL REFERENCES pelicula(id_pelicula),
    id_sala BIGINT NOT NULL REFERENCES sala(id_sala),
    fecha_hora TIMESTAMPTZ NOT NULL,
    precio_base NUMERIC(12,2) NOT NULL CHECK (precio_base >= 0),
    activa BOOLEAN NOT NULL DEFAULT TRUE,
    id_admin_creador BIGINT REFERENCES administrador(id_admin),
    UNIQUE (id_funcion, id_sala)
);
CREATE TABLE compra (
    id_compra BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_cliente BIGINT NOT NULL REFERENCES cliente(id_cliente),
    id_taquillero BIGINT REFERENCES taquillero(id_taquillero),
    creada_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    total NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total >= 0),
    estado VARCHAR(15) NOT NULL DEFAULT 'pendiente'
      CHECK (estado IN ('pendiente','pagada','fallida','cancelada','reembolsada'))
);
CREATE TABLE pago (
    id_pago BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_compra BIGINT NOT NULL REFERENCES compra(id_compra),
    referencia_externa VARCHAR(120) UNIQUE,
    metodo VARCHAR(30) NOT NULL,
    monto NUMERIC(12,2) NOT NULL CHECK (monto >= 0),
      estado VARCHAR(15) NOT NULL
      CHECK (estado IN ('pendiente','aprobado','rechazado','reembolsado')),
    procesado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE entrada (
      id_entrada INT PRIMARY KEY,
      funcion VARCHAR(40),
      asiento INT,
      id_sala BIGINT,
      numero_asiento VARCHAR(12),
      estado VARCHAR(15) NOT NULL DEFAULT 'emitida' CHECK (estado IN ('emitida', 'usada',
'cancelada')),
      id_compra BIGINT REFERENCES compra(id_compra),
      CONSTRAINT fk_asiento
      FOREIGN KEY (id_sala, numero_asiento) REFERENCES asiento(id_sala, numero),
      dia TIMESTAMP,
      hora TIME,
```

```
      tarifa DECIMAL,
      id_taquillero INT,
      CONSTRAINT fk_taquillero
      FOREIGN KEY (id_taquillero) REFERENCES taquillero(id_taquillero),
      id_funcion INT,
      CONSTRAINT fk_funcion
      FOREIGN KEY (id_funcion) REFERENCES funcion(id_funcion),
      CONSTRAINT fk_sala
      FOREIGN KEY (id_sala) REFERENCES sala(id_sala)
);
CREATE UNIQUE INDEX ux_entrada_funcion_asiento_vigente
  ON entrada (id_funcion, id_sala, numero_asiento)
  WHERE estado IN ('emitida','usada');
CREATE TABLE comprobante (
    id_qr BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_entrada BIGINT NOT NULL UNIQUE REFERENCES entrada(id_entrada),
    codigo_qr UUID NOT NULL UNIQUE,
    creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE bloqueo_asiento (
    id_bloqueo BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_cliente BIGINT NOT NULL REFERENCES cliente(id_cliente),
    id_funcion BIGINT NOT NULL,
    id_sala BIGINT NOT NULL,
    numero_asiento VARCHAR(12) NOT NULL,
    creado_en TIMESTAMPTZ NOT NULL DEFAULT NOW(),
      vence_en TIMESTAMPTZ NOT NULL DEFAULT (NOW() + INTERVAL '2 minutes'),
    estado VARCHAR(12) NOT NULL DEFAULT 'activo'
      CHECK (estado IN ('activo','liberado','comprado','expirado')),
    CHECK (vence_en > creado_en),
    FOREIGN KEY (id_funcion, id_sala) REFERENCES funcion(id_funcion, id_sala),
    FOREIGN KEY (id_sala, numero_asiento) REFERENCES asiento(id_sala, numero)
);
CREATE TABLE transaccion_log (
    id_log BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_compra BIGINT REFERENCES compra(id_compra),
    id_pago BIGINT REFERENCES pago(id_pago),
    tipo_evento VARCHAR(60) NOT NULL,
    detalle JSONB,
    ocurrido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE asiento_auditoria_log (
    id_log BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_funcion BIGINT REFERENCES funcion(id_funcion),
    id_sala BIGINT REFERENCES sala(id_sala),
    numero_asiento VARCHAR(12),
    id_cliente BIGINT REFERENCES cliente(id_cliente),
    tipo_evento VARCHAR(40) NOT NULL,
    ocurrido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE acceso_puerta_log (
    id_log BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_entrada BIGINT REFERENCES entrada(id_entrada),
    id_operador BIGINT REFERENCES operador(id_operador),
    dispositivo VARCHAR(80) NOT NULL,
    resultado VARCHAR(20) NOT NULL CHECK (resultado IN ('exitoso','rechazado')),
    motivo VARCHAR(120),
    ocurrido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE admin_operacion_audit (
    id_log BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_admin BIGINT NOT NULL REFERENCES administrador(id_admin),
    rol VARCHAR(40) NOT NULL DEFAULT 'administrador',
    direccion_ip INET,
    operacion VARCHAR(80) NOT NULL,
    tabla_afectada VARCHAR(80) NOT NULL,
    registro_id BIGINT,
    detalle JSONB,
    ocurrido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.3.2 Inserción de datos de prueba:

Para diferenciar los id de cada tabla, se les asigna un número fijo de acuerdo a la entidad:

Tabla	Valor fijo (base)
Cliente	10X
Taquillero	20X
Operador	30X
Contador	40X
Administrador	50X
Entrada	60X
Función	70X
Comprobante	80X
Sala	90X

```
INSERT INTO clientes (id_cliente, nombre, grupo_edad) VALUES
(101, ‘Juan’, ‘adulto’),
(102, ‘Pedro’, ‘niño’),
(103, ‘Luis’, ‘mayor’),
(104, ‘Ana’, ‘estudiante’),
(105, ‘María’, ‘adulto’);

INSERT INTO taquillero (id_taquillero, nombre) VALUES
(201, ‘taq1’),
(202, ‘taq2’);

INSERT INTO operador (id_operador, nombre) VALUES
(301, ‘op1’),
(302, ‘op2’);

INSERT INTO contador (id_contador, nombre) VALUES
(401, ‘cont1’),
(402, ‘cont2’);

INSERT INTO administrador (id_admin, nombre) VALUES
(501, ‘admin1’),
(502, ‘admin2’);

INSERT INTO pelicula (id_pelicula, titulo, sinopsis, clasificacion, duracion_min, activa)
VALUES
(601, 'Interestelar', 'Un grupo de astronautas viaja a través de un agujero de gusano en
busca de un planeta habitable para salvar a la humanidad.', 'PG-13', 169, TRUE),
```

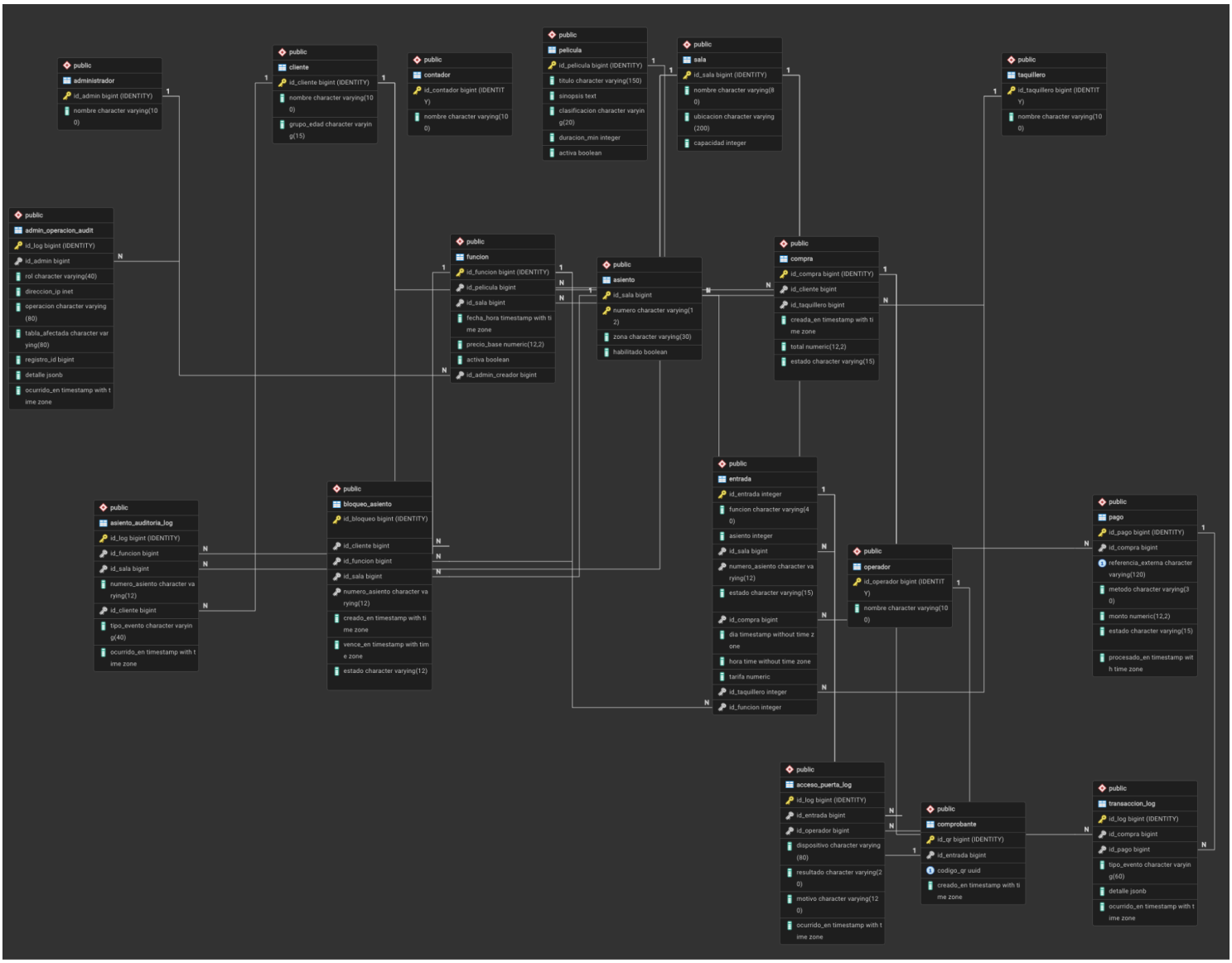
```
(602, 'Avatar: El camino del agua', 'Jake Sully y su familia deben abandonar su hogar y
buscar refugio entre los pueblos oceánicos de Pandora.', 'PG-13', 192, TRUE),
(603, 'Oppenheimer', 'La historia de J. Robert Oppenheimer y su participación en el
desarrollo de la bomba atómica durante la Segunda Guerra Mundial.', 'R', 180, TRUE);

INSERT INTO sala (id_sala, nombre, ubicacion, capacidad)VALUES
(1, 'Sala IMAX 1', 'San José', 20),
(2, 'Sala IMAX 2', 'Los Angeles', 20),
(3, 'Sala IMAX 3', 'Amsterdam', 20);


INSERT INTO asiento (id_sala, numero, zona, habilitado)
SELECT
    s.id_sala,
    f.fila || n.numero,
    CASE
        WHEN f.fila = 'D' THEN 'VIP'
        ELSE 'general'
    END,
    TRUE
FROM sala s
CROSS JOIN (VALUES ('A'), ('B'), ('C'), ('D')) AS f(fila)
CROSS JOIN generate_series(1, 5) AS n(numero)
WHERE s.id_sala IN (1, 2, 3);


INSERT INTO funcion (id_funcion, id_pelicula, id_sala, fecha_hora, precio_base, activa,
id_admin_creador) VALUES
(701, 601, 1, '2026-11-05 18:00:00-06', 4500, TRUE, 501),
(702, 602, 2, '2026-11-05 19:00:00-06', 5000, TRUE, 501),
(703, 603, 3, '2026-11-06 20:00:00-06', 5500, TRUE, 501);
```

4. Modelo Relacional



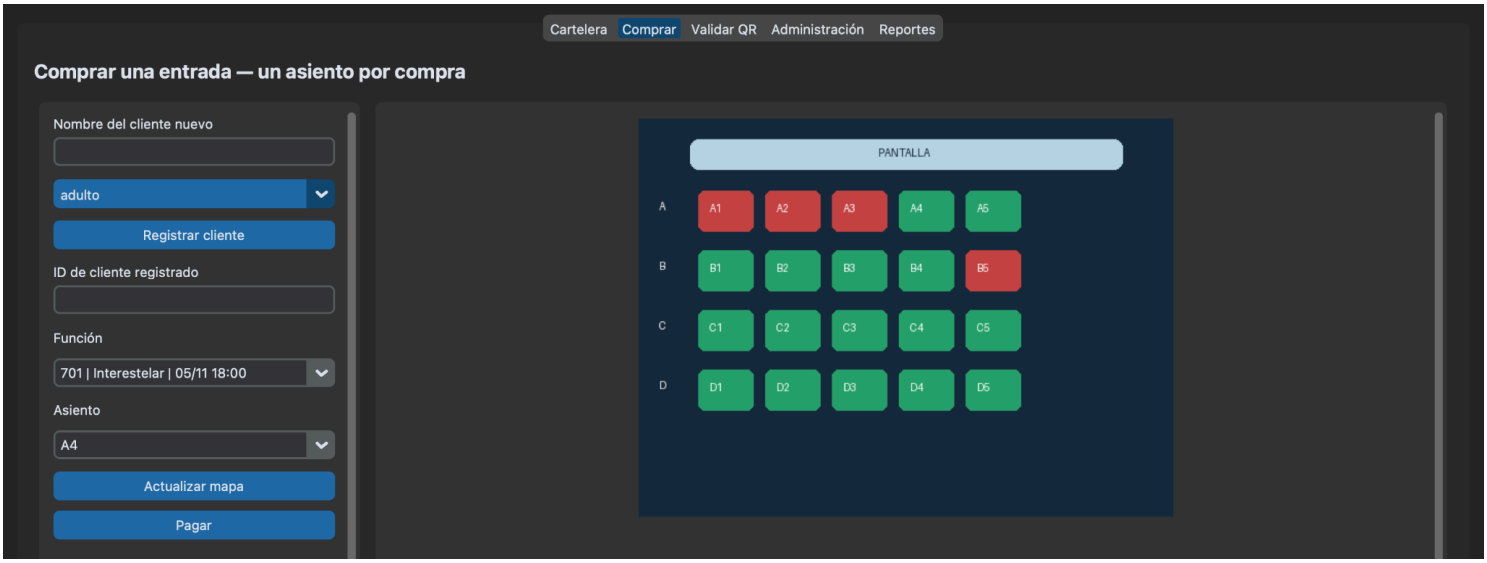
5. Interfaz gráfica

5.1 Vista de cartelera y exploración



Al ejecutar el programa la primera pestaña que se observa es la pestaña de cartelera. En esta se contienen los títulos, las sinopsis, la hora, fecha y el precio de la entrada.

5.2 Mapa interactivo de la sala



La siguiente pestaña corresponde a la compra de tiquetes y selección de asientos. En la barra lateral está el menú para registrar un cliente que no estaba dentro de la base de datos, además se agrega su grupo etario. También está la opción de introducir el ID de un cliente registrado en la base de datos. Una vez esos datos estén debidamente introducidos, se puede proceder a la compra y selección. Se selecciona la función que se quiera ver según la cartelera disponible y, seguidamente se escogen los asientos según su disponibilidad. Los asientos verdes corresponden a los asientos disponibles y los rojos a los asientos comprados. Cabe recalcar que el sistema solo permite una venta por usuario.

5.3 Módulo de pago

Este módulo se encuentra dentro de la misma pestaña. Ya que este es un pago simulado, no se pide información sobre datos de la tarjeta, ya que estos datos no deberían de almacenarse.

5.4 Pantalla de confirmación y entrada digital

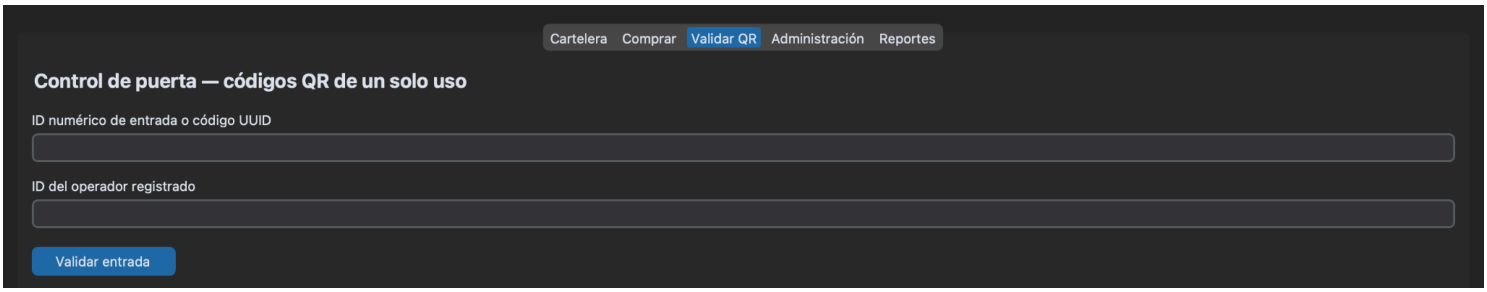


Una vez que el pago esté aprobado, el sistema abrirá una nueva ventana con el código QR correspondiente a la entrada comprada. Esta confirmación reitera la función que se compró y el ID numérico para el QR, este ID es importante ya que con este se registra la entrada como usada.

5.5 Panel de administración (*dashboard*)

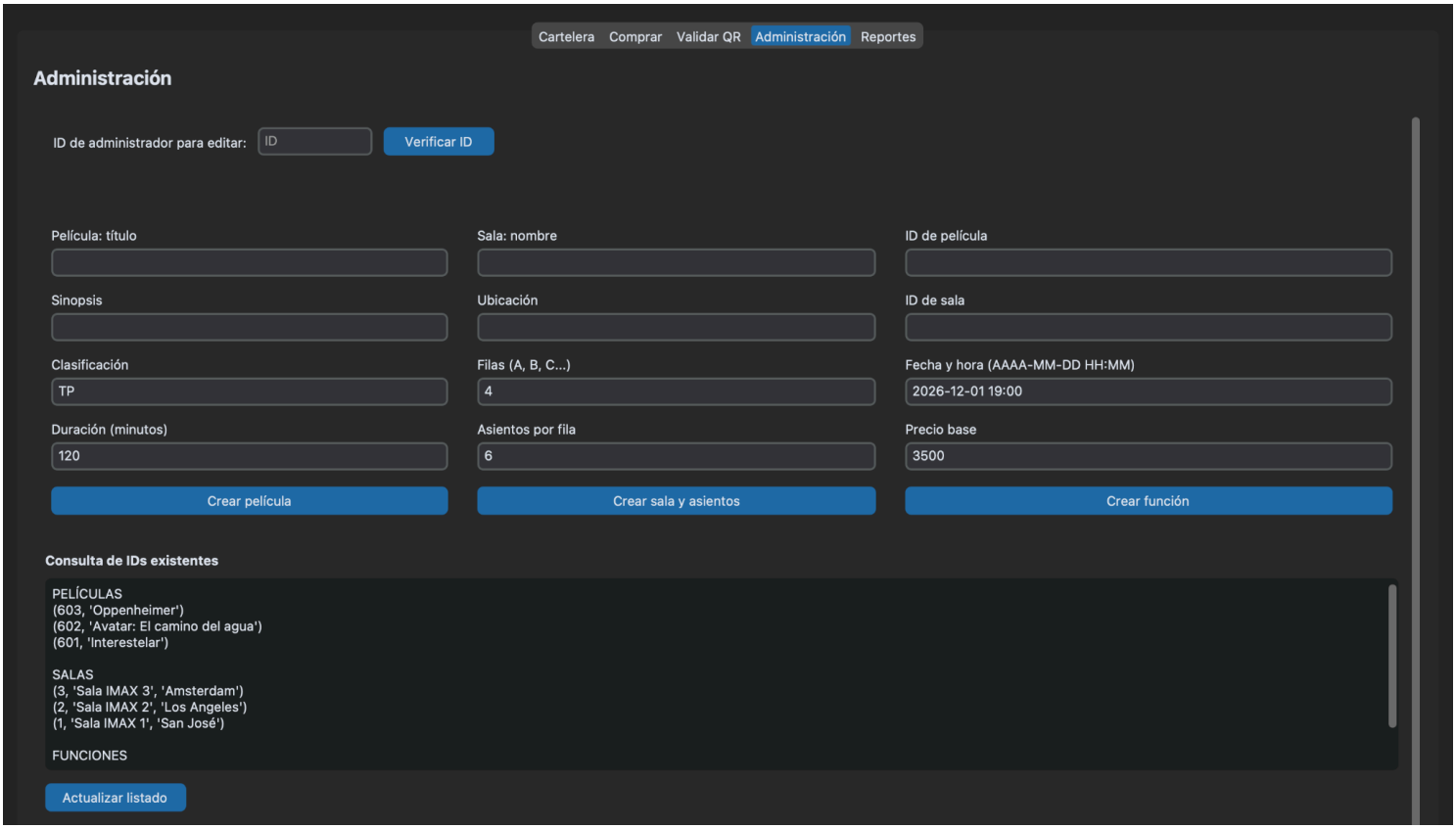
El panel de administración está dividido en tres pestañas diferentes: validación de QR, administración y reportes.

5.5.1. Validación de QR



En esta ventana se valida el código QR. Se introduce el ID numérico del QR y un operador registrado debe validar que la entrada no se haya usado anteriormente. Si el ID del operador no está dentro de la base de datos, el QR no se puede validar.

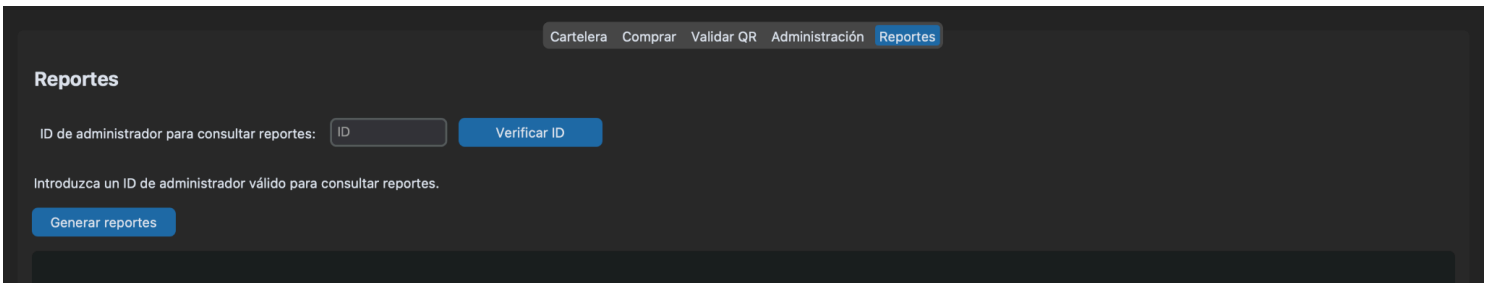
5.5.2. Administración



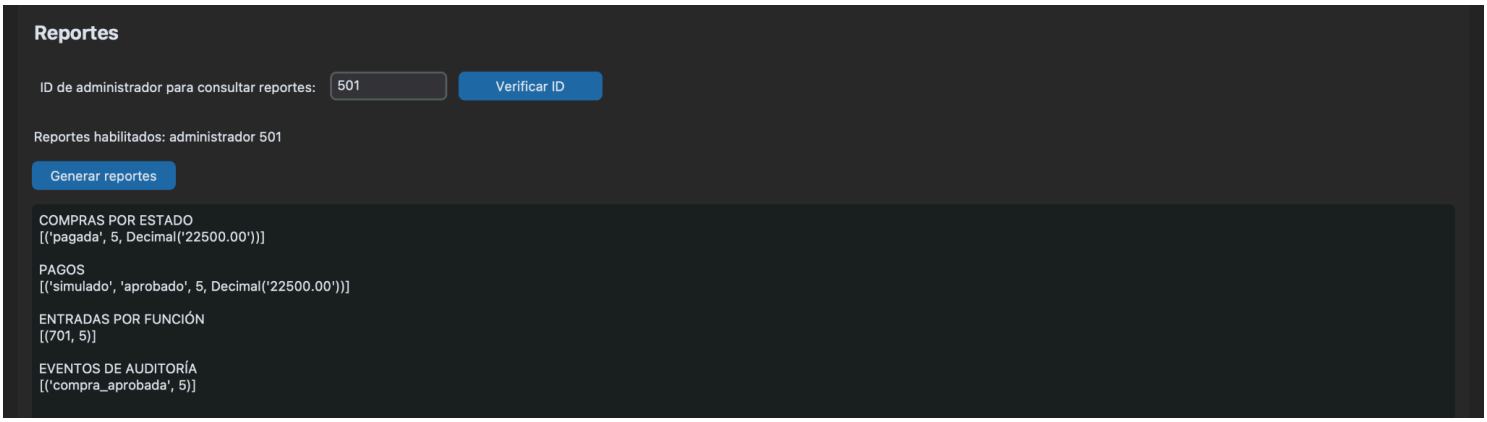
En esta pestaña solo pueden editarla las personas administradoras que estén registradas en la base de datos. Si el ID del administrador no es válido, no se puede proseguir con la edición de los campos de texto. En la primera columna la persona administradora puede agregar una nueva película (que conlleva agregar el título, la sinopsis, la clasificación y la duración). En la segunda columna la persona administradora puede agregar una nueva sala (esto conlleva el nombre, su ubicación, la cantidad de filas y los asientos que tiene cada fila). En la tercera columna se crea la función (que contiene el ID de la película, que es brindado cuando se crea la película, el ID de la sala, que es brindado cuando se crea la sala, y se establece la hora y el precio del boleto).

Adicionalmente, en la parte inferior de la pantalla se pueden consultar los IDs existentes en la base de datos y a que corresponde cada ID.

5.5.3. Reportes



Para poder visualizar los reportes primero se debe introducir un ID de persona administradora.



Una vez se introduzca un ID válido se debe presionar ‘Generar reportes’. El primer desglose es las compras por estado: 5 entradas se han comprado y estas ventas equivalen a ₡22500. Los pagos: estos han sido simulados, entonces tienen la etiqueta de ‘simulado’, también han sido aprobados y el total de entradas compradas equivalen a ₡22500. Las entradas por función: el primer número que se muestra es el ID de la función y el siguiente número es la cantidad de entradas que esa función ha vendido. Por último se encuentran los eventos de auditoría: estos muestran los estados de pago, que en nuestro caso no hay pagos rechazados, y la cantidad de ventas que han habido.

Bibliografía

Google Cloud (s.f). *Calculadora de precios de Google Cloud.* https://cloud.google.com/products/calculator?hl=es&dl=CjhDaVF3WTJabE1qaGtNeTFtTkRJMkxUUTROV1V0T1dWak9DMDNOV05tWXpJNU1UWTRNVEFRQVE9PRBLGiRBMjg4QTE3RS0zNjk1LTRDODEtQjE3Qi1CRUUzRDFBRTc0Qjk

Anexos

1. Link de acceso al repositorio: https://github.com/Gabo3108/Proyecto-2---Bases-de-datos.git

2. Declaración de uso de IA:

Campo	Descripción
Ejercicio	Documento de código Python (configuración de la interfaz)
Herramientas utilizadas	ChatGPT-5 Luna
Etapa del trabajo	Comprensión del código python
Descripción breve del uso	Se le solicitó a la IA explicar la plantilla del código para poder realizar las operaciones necesarias dentro de la interfaz del sistema
Declaración de autoría	"Declaro que comprendo y puedo explicar la totalidad del trabajo entregado, y que la IA fue utilizada únicamente como apoyo, no como sustituto de mi trabajo."

Campo	Descripción
Ejercicio	Documento de código Python (ayuda con la interfaz de los asientos)
Herramientas utilizadas	ChatGPT-5 Luna
Etapa del trabajo	Ejecución del código python
Descripción breve del uso	Se le solicitó a la IA colaboración y explicación para realizar el módulo de la selección de los asientos
Declaración de autoría	"Declaro que comprendo y puedo explicar la totalidad del trabajo entregado, y que la IA fue utilizada únicamente como apoyo, no como sustituto de mi trabajo."

Campo	Descripción
Ejercicio	Documento de código Python (corrección de errores en el código)
Herramientas utilizadas	ChatGPT-5 Luna
Etapa del trabajo	Ejecución del código python
Descripción breve del uso	Se le solicitó a la IA colaboración y explicación para corregir el código que se tenía hasta el momento
Declaración de autoría	"Declaro que comprendo y puedo explicar la totalidad del trabajo entregado, y que la IA fue utilizada únicamente como apoyo, no como sustituto de mi trabajo."